AgentCgroup

AI Agent Resource Characterization and OS-Level Control

Yusheng Zheng (yzhen165@ucsc.edu)
PhD Student

Roadmap

- Motivation & Key Findings
- Resource Characterization (144 tasks)
- Agent vs. Cloud & Three Mismatches
- System Design: eBPF + Bash Wrapper + Negotiation
- Evaluation & Takeaways

Motivation

Why study OS resources of AI agents?

AI coding agents execute in **multi-tenant cloud sandboxes** — but their OS-level resource behavior is **poorly understood**.

What we found (144 SWE-bench tasks)

- **OS-level overhead = 56~74%** of end-to-end latency; LLM reasoning only 26~44%
- **Memory, not CPU**, limits multi-tenant concurrency density
- Demands vary **20x across tasks**, **1.8x across runs** of the same task

Characterization: Setup

144 SE tasks from SWE-rebench benchmark, with claude code

Config	Model	Tasks
Local GPU	GLM-4.7-Flash	111
Cloud API	Claude Haiku 4.5	33

Hardware: Intel Core Ultra 9 285K, 24 cores, 128GB DDR5

Environment: Podman containers, Linux 6.15, cgroup v2, 1s sampling

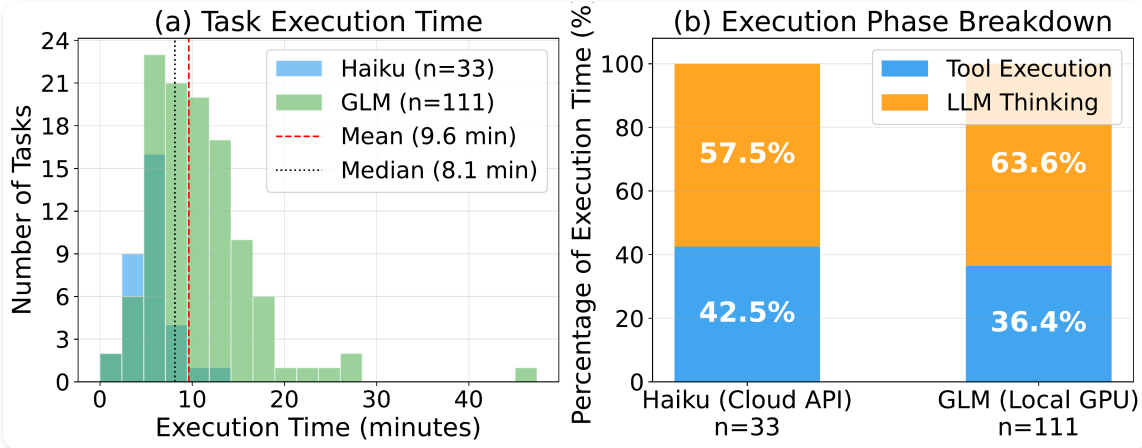
What We Measure

- **Execution model**: granularity at which resources vary
- **Temporal dynamics**: how fast controls must react
- **Cross-task variance**: whether demands can be predicted

Container Images

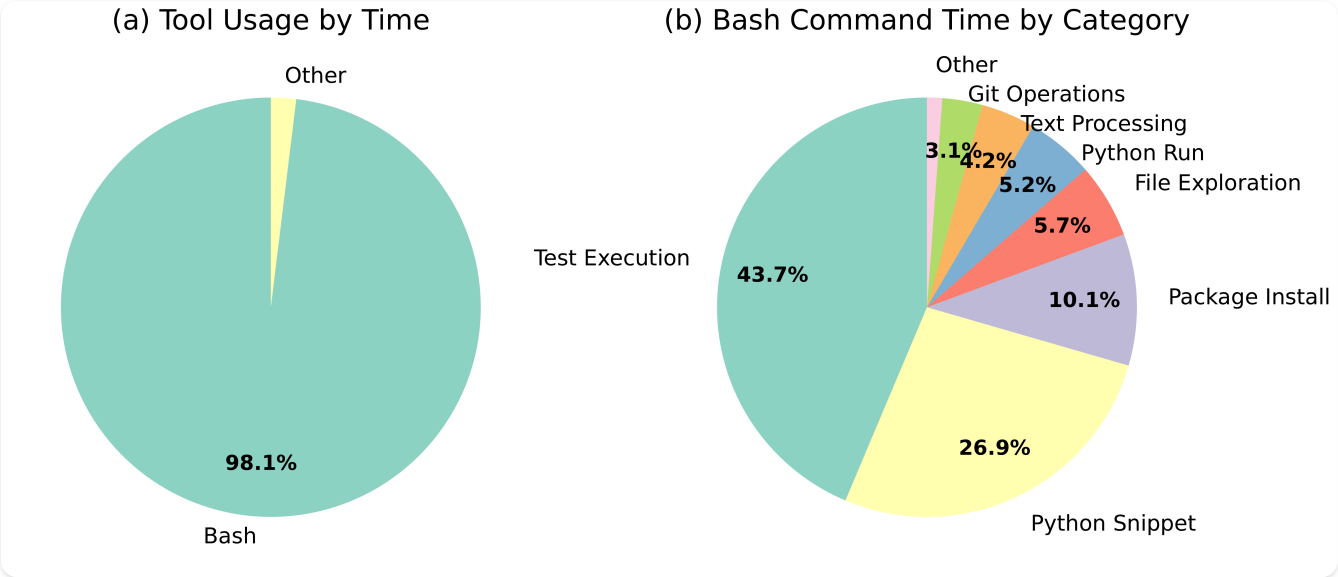
- **Average 3.5 GB** (range 2.9~17.3 GB)
- 7x larger than microservices, 70x larger than serverless

Execution Model

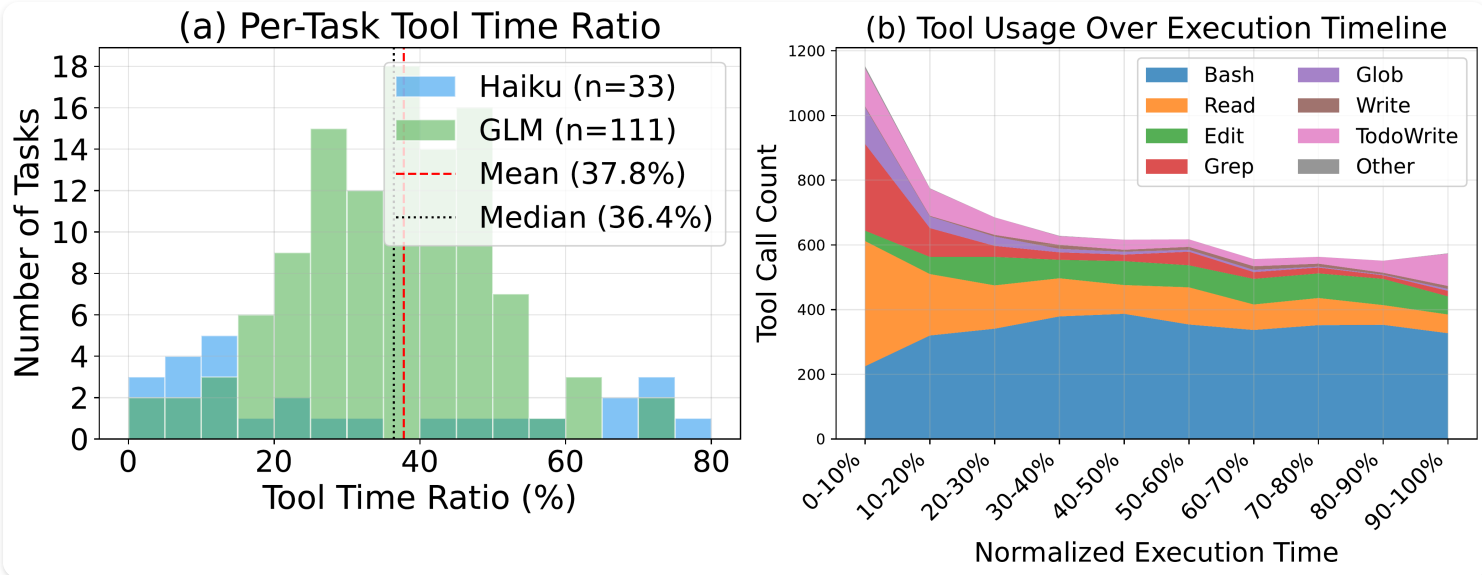


- **LLM reasoning accounts for 26~44% of end-to-end task latency; the remainder is consumed by tool execution (~40% of active time) and initialization (29~45%)**

Tool Execution: What Dominates?

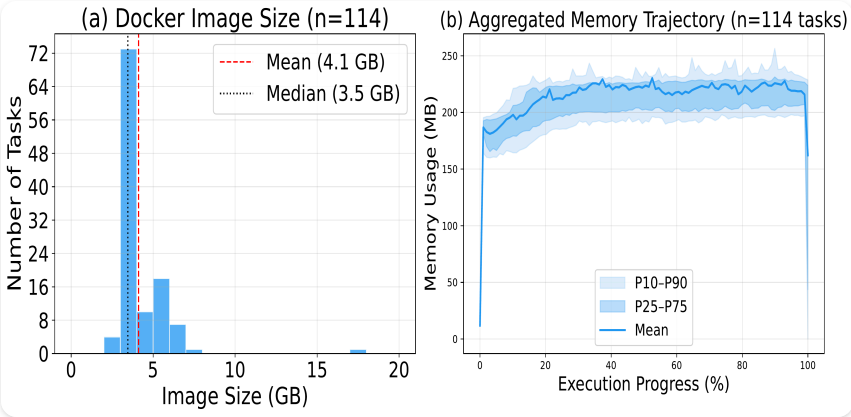


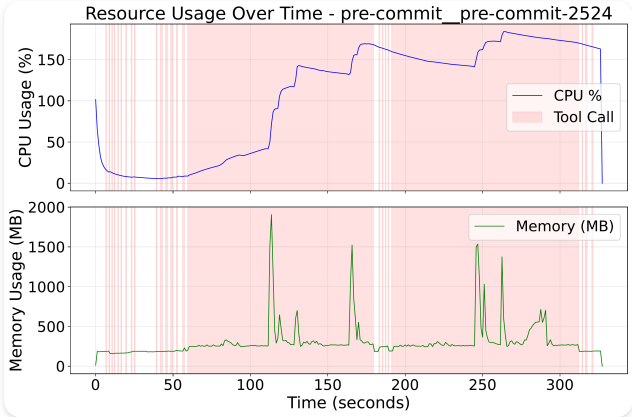
Tool Execution: Temporal Pattern



- Tool calls follow a "**understand, modify, verify**" pattern: Read in first 30%, Bash peaks at 40~80%

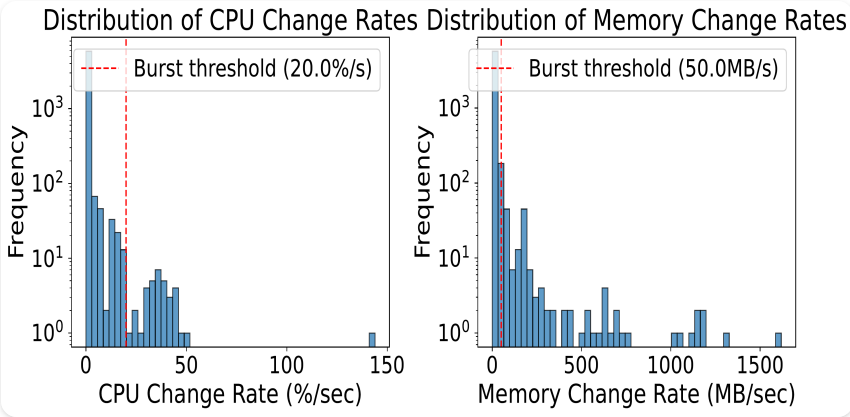
Resource Dynamics

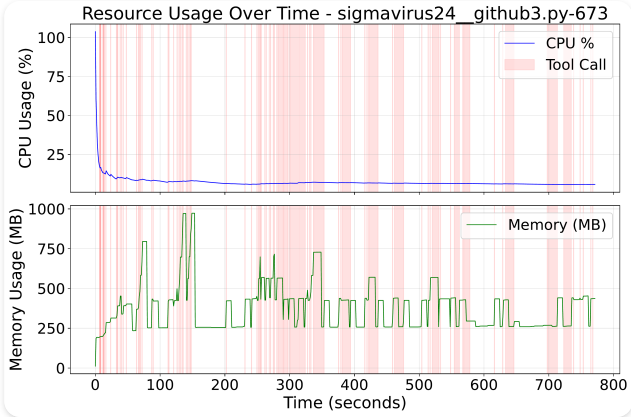




- **Resource consumption exhibits a two-layer structure: a ~185 MB framework baseline plus tool-call bursts**
- **What the tool *does* matters, not which tool** (e.g., Bash vs. Read): 13.7x peak memory difference
- **98.5% of memory bursts occur during tool calls**

Burst Dynamics & Variability





- **Resource bursts last 1~2 seconds with peak-to-average ratio up to 15.4x**
- **85%~97% of tasks contain retry loops with progressive memory accumulation**
- **CPU-memory correlation varies by task (−0.84 to +0.50); co-directional change cannot be assumed**

Agent vs. Cloud Workloads

Dimension	Serverless/FaaS	Microservices	Batch/HPC	**AI Coding Agent**
Execution duration	100ms–2s	Long-running	Minutes–hours	**5–11 minutes**
Container image	~50 MB	100 MB–1 GB	1–10 GB	**2.9–17.3 GB (med. 3.5)**
Memory footprint	128–512 MB	Steady ~1 GB	Scales with data	**185 MB idle, peaks 2–4 GB**
Memory peak/avg	~1.5×	2–3×	~1×	**15.4×**
CPU utilization	Brief spike	10–40%	80–100%	**<13% avg, peaks >175%**
Determinism	Deterministic	Mostly deterministic	Deterministic	**1.8× variance same task**
Resource pattern	Flat	Steady + daily cycle	Stable rise	**Burst-silence alternating**
Termination cost	Just retry	Can migrate	Lose progress	**Lose all LLM context**

Three Mismatches

	Static Limits	Reactive Control	Predictive Scaling
Tools	mem.max/high, cpu.max; K8s QoS	PSI; oomd; TMO	VPA; Autopilot
Assumes	Known peak; stable demand	Gradual pressure; kill OK	Repeatable; history valid
Agent	15.4× peak/avg; tool-semantic	1–2s burst; unpredictable	1.8× variance; kill = lose context
Mismatch	**Granularity**	**Responsiveness**	**Adaptability**

- **Granularity**: `memory.max` at peak wastes >90%; at average → OOM
- **Responsiveness**: 3 GB/s bursts too fast for user-space tools
- **Adaptability**: Kill = triple penalty (cold start + lost context + re-execution)

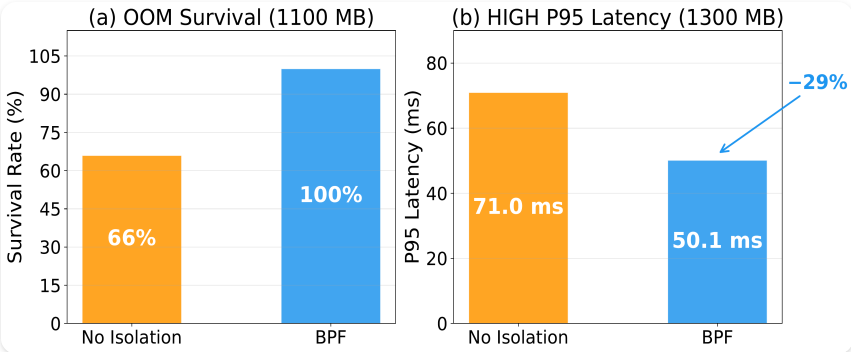
System Design

1. **Fine-Grained Resource Domains** → *Granularity*: bash wrapper → per-tool-call child cgroups

2. **In-Kernel eBPF** → *Responsiveness*: `sched_ext` + `memcg_bpf_ops`, **throttle** → **freeze** → **never kill**

3. **Bidirectional Negotiation** → *Adaptability*: agent hints upward, NL feedback downward

Evaluation



3 concurrent agents, replayed traces at 50x

Tight memory: OOM survival 66% → **100%**; HIGH overhead only +2.8%

Moderate memory: P95 latency **29% better** (71ms → 50ms)

Wrapper < 5ms/call · Throttle precision 2.3% · Hint accuracy 100%

Throttle-don't-kill eliminates OOM deaths with < 3% overhead

Limitations & Future Work

Limitations

- Requires patched kernel (`memcg_bpf_ops` under review)
- Evaluated with 3 agents; production runs 50-100+
- Bash-only interception

Future Work

- Larger-scale multi-tenant evaluation
- Upstream `memcg_bpf_ops` into mainline Linux
- Cross-agent scheduling (OpenClaw, Claude Code, Codex)
- Predictive hints from tool-call history

Thank You

Questions?

References

AgentCgroup Paper (arXiv:2602.09345) · AgentCgroup GitHub